PAWSv3 Background Generator & NUCU Co-Processor Documentation

The PAWSv3 background generator is a dedicated hardware display layer capable of rendering 16 distinct procedural background patterns. It is tightly integrated with the "NUCU" (Background Co-Processor), a specialized processing unit that executes custom assembly instructions synchronized to the video phase, allowing for dynamic raster effects, parallax scrolling, and procedural generation without taxing the main CPU.

Important Color Note: The background layer serves as the absolute backplate of the display stack; therefore, it does not support transparency. If color 64 (0x40) is applied to the background layer, it will render as DKRED (Dark Red) rather than transparent.

1. Background Modes

The background generator supports 16 hardware-accelerated rendering modes (0 to 15), defined in PAWSdefinitions_2.h. These modes manipulate the primary color (b_colour) and alternate color (b_alt) based on screen coordinates.

- **0: BKG_SOLID** - Renders a solid screen using the primary color.
- **1-4: BKG_CHKBRD_1 to BKG_CHKBRD_4** - Renders checkerboard patterns of varying sizes using XOR logic on the X and Y coordinates.
- **5: BKG_QUARTERS** - Divides the screen into four quadrants alternating between primary and alternate colors.
- **6: BKG_5050_V** - Splits the screen vertically (left/right).
- **7: BKG_5050_H** - Splits the screen horizontally (top/bottom).
- **8: BKG_RAINBOW** - Renders a hardware-generated rainbow gradient driven by the Y coordinate.
- **9: BKG_SNOW** - Renders a pseudo-random scrolling starfield/snow effect.
- **10: BKG_STATIC** - Renders random static noise using the hardware random number generator.
- **11: BKG_HATCH** - Renders a crosshatch pattern.
- **12: BKG_LSLOPE** - Renders diagonal stripes sloping left.
- **13: BKG_RSLOPE** - Renders diagonal stripes sloping right.

- **14: BKG_VSTRIPE** - Renders vertical stripes.
- **15: BKG_HSTRIPE** - Renders horizontal stripes.

2. NUCU Co-Processor Architecture

The NUCU Co-Processor runs in parallel with the main CPU and executes its own instruction set out of dedicated block RAM.

NUCU Specifications

- **Program Memory:** 1024 instruction blocks.
- **Data Memory:** 64 word memory space available for user data/palettes.
- **Stack:** 16-level hardware stacks for both data (dstack) and return addresses (rstack).

Hardware Registers

The NUCU environment provides distinct registers for reading system states and writing display states.

Read-Only System Inputs:

- CU_RB: VBLANK status (0 = Active, 1 = VBLANK).
- CU_RX: Current X coordinate (0-639).
- CU_RY: Current Y coordinate.
- CU_RC: CPU Input (User-defined value passed from the RISC-V CPU).
- CU_RNG: Random Number Generator output.

General Purpose Registers:

- CU_R0, CU_R1, CU_R2, CU_R3: Standard 10-bit read/write registers for ALU operations.

Write-Only Display Outputs:

- CU_BM: Background Mode.
- CU_BC: Background Primary Color.
- CU_BA: Background Alternate Color.

3. NUCU Instruction Set

The NUCU assembler supports the following opcodes, parsed via the copper_assemble() C library function.

ALU Operations

ALU operations modify a register based on a secondary register or literal value.

- SET Reg, Val: Sets the target register to the value.
- ADD Reg, Val: Adds value to the register.
- SUB Reg, Val: Subtracts value from the register.
- AND Reg, Val: Bitwise AND.
- OR Reg, Val: Bitwise OR.
- XOR Reg, Val: Bitwise XOR.
- MUL Reg, Val: Multiplies the register by the value.
- SHR Reg, Val: Shifts the register right by the value.
- SHL Reg, Val: Shifts the register left by the value.
- ABS Reg, Val: Stores the two's complement absolute value.
- MIN Reg, Val: Stores the minimum of the two values.
- MAX Reg, Val: Stores the maximum of the two values.

Branching & Conditionals

Conditional instructions skip the *next* instruction if the condition is met.

- SEQ Reg, Val: Skip next instruction if Reg == Val.
- SNE Reg, Val: Skip next instruction if Reg != Val.
- SLT Reg, Val: Skip next instruction if Reg < Val.
- SGE Reg, Val: Skip next instruction if Reg >= Val.
- SLE Reg, Val: Skip next instruction if Reg <= Val.
- SGT Reg, Val: Skip next instruction if Reg > Val.

Flow Control & Memory

- JMP Addr: Jump to program address.
- JSR Addr: Jump to subroutine (pushes return address to rstack).
- RET: Return from subroutine (pops rstack).
- PUSH Val: Push literal or register to dstack.
- POP Reg: Pop value from dstack into target register.

- LFM Reg, Addr: Load from NUCU memory into register.
- STM Reg, Addr: Store register into NUCU memory.

Video Synchronization Waits

Wait instructions stall the NUCU pipeline until a specific video state is reached.

- WAITACTIVE: Stalls until the active display area begins (waits for VBLANK to end).
- WAITVB: Stalls until VBLANK begins.
- WAITX Val: Stalls until the X coordinate reaches the designated value.
- WAITY Val: Stalls until the Y coordinate reaches the designated value.

4. C Library API (PAWSlibrary.h)

The PAWS C library provides several functions to interact with the background generator and NUCU co-processor.

Standard Background Control

- void set_background(unsigned char colour, unsigned char altcolour, unsigned char backgroundmode); Sets the primary color, alternate color, and background pattern mode. Calling this function automatically halts the NUCU co-processor.

NUCU Co-Processor Control

- void copper_startstop(unsigned char status); Starts (status = 1) or stops (status = 0) the co-processor execution. This command waits for VBLANK to ensure safe synchronization.
- void set_copper_cpuinput(unsigned short value); Pushes a 10-bit value to the co-processor, accessible via the CU_RC register.
- unsigned short get_copper_cpuoutput(void); Retrieves the value currently output by the co-processor.
- void copper_init_memory(unsigned short *memory, unsigned int number); Copies an array of 10-bit values into the co-processor's memory space.

NUCU Assembler

- void copper_assemble(unsigned short address, char *assembly); Parses a string of NUCU assembly code and writes the compiled instruction to the specific block address.

- void copper_assemble_output(unsigned short address, char *assembly); Compiles the instruction and prints the assembly text to the Character Map layer for debugging.
- void copper_assemble_comment(unsigned short address, char *assembly, char *comment); Compiles the instruction and prints the assembly text alongside an inline comment to the Character Map layer.

5. Memory-Mapped I/O Registers

For low-level driver development, the background generator can be controlled directly via the hardware bus at base address 0xd000.

Address	Register Name	Description	Size
0xd000	BACKGROUND_COLOUR	Primary pattern color	8-bit
0xd002	BACKGROUND_ALTCOLOUR	Alternate pattern color	8-bit
0xd004	BACKGROUND_MODE	Pattern mode (0-15)	8-bit
0xd006	BACKGROUND_COPPER_STARTSTOP	Run/Stop status (0/1)	8-bit
0xd008	BACKGROUND_COPPER_CPUINPUT	CPU -> NUCU RC Register	16-bit
0xd008	BACKGROUND_COPPER_CPUOUTPUT	NUCU -> CPU Register (Read-only)	16-bit
0xd00a	BACKGROUND_COPPER_PROGRAM	Instruction Write Trigger	8-bit
0xd00c	BACKGROUND_COPPER_ADDRESS	Instruction Write Address	16-bit

Address	Register Name	Description	Size
0xd010	BACKGROUND_COPPER_OP	Raw Instruction Data	32-bit
0xd016	BACKGROUND_COPPER_MEMADDR	Data Memory Address	8-bit
0xd018	BACKGROUND_COPPER_MEMVALUE	Data Memory Value	16-bit

Note: All Memory-Mapped I/O locations are defined as volatile pointers in PAWS.h.